Git & GitHub in 5 Days

A practical **10-hour learning sprint**: 5 days, 2 hours per day. Every topic follows the pattern **Learn this → Do this**. The goal is to become comfortable with the everyday Git workflow and use GitHub to publish work, collaborate, and review changes.

How to use this roadmap

1. Create a GitHub repository named `git-github-5-day-roadmap` (or any name you prefer).
2. Add this `README.md` and `tracker.html` to the repository.
3. Open `tracker.html` in your browser and tick tasks as you complete them. Its progress is saved in that browser on that device.
4. Tick the matching boxes in this README and commit your progress to GitHub at the end of each day.
5. Keep your practice commands and mistakes in a `notes/` folder. Learning from the errors you fix is part of the process.

> **Tracking note:** the HTML tracker uses your browser's local storage. Its checkmarks do not automatically sync to GitHub or to another device. Use the Markdown checklist in this README for GitHub-visible progress. You can also export a JSON backup from the HTML tracker.

Before Day 1

- ☐ Install Git and confirm it works: `git --version`
- ☐ Create or sign in to your GitHub account.
- ☐ Configure the name and email that should appear on your

commits:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
```

- ☐ Check the configuration: `git config --global --list`
- ☐ Create a practice folder and open a terminal in it.

Use an email address associated with your GitHub account, or a GitHub-provided `noreply` address if you prefer to keep your email private.

Day 1 — Git foundations and your first GitHub push

Outcome: understand the basic Git workflow, make commits, and publish a repository to GitHub.

Time	Learn this → Do this
0:00–0:20	**Git vs GitHub; repository, working directory, staging area, commit.** → Explain the four terms in your own words. Check `git --version` and configure your identity.
0:20–0:45	**`init`, `status`, `add`, `commit`.** → Run `git init`, create `README.md`, stage it with `git add README.md`, and make your first commit.
0:45–1:05	**`diff`, `diff --staged`, `log`, `show`.** → Edit the README, inspect unstaged and staged changes, commit them, and inspect the commit history.
1:05–1:25	**`.gitignore` and tracked vs untracked files.** → Add common generated files (for example, `.DS_Store`, `*.log`, and `node_modules/`) to `.gitignore`; verify with `git status`.

1:25–1:45	**Remote repositories; `remote`, `push`.** → Create an empty GitHub repository, connect it using `git remote add origin <repo-url>`, and push with `git push -u origin main`.
1:45–2:00	**Review and recall.** → From memory, explain `status → add → commit → push`; run `git status` and `git log --oneline`.

Day 1 checklist

- ☐ I can describe the working directory, staging area, commit, and remote.
- ☐ I initialized a repository and made at least 3 meaningful commits.
- ☐ I used `status`, `add`, `diff`, `diff --staged`, `log`, and `show`.
- ☐ I created a `.gitignore` and checked that ignored files are not staged.
- ☐ I pushed my repository to GitHub.

Deliverable: a GitHub repository with a useful README, `.gitignore`, and a visible commit history.

Day 2 — Branches, merges, and conflicts

Outcome: work safely on separate branches and combine changes.

Time	Learn this → Do this
0:00–0:20	**Why branches exist; `branch`, `switch`.** → Run `git branch`, create `feature/profile`, and switch to it.
0:20–0:45	**Branch-based workflow.** → Add a profile section or file on `feature/profile`; make 2 small commits.
0:45–1:05	**`merge` and fast-forward merges.** → Switch to `main`, merge the feature branch, and inspect the history.

1:05–1:30	**Merge conflicts and conflict markers.** → Create two branches that edit the same line differently, merge them, resolve the conflict, then stage and commit the resolution.
1:30–1:45	**`branch -d`, `branch -a`, `log --oneline --graph --all`.** → Delete the merged local branch and inspect all branches in the graph.
1:45–2:00	**Review.** → Create one more short-lived branch, merge it, and explain what caused the conflict and how you resolved it.

Day 2 checklist

- ☐ I created and switched between branches.
- ☐ I made commits on a feature branch without changing `main` directly.
- ☐ I merged a feature branch into `main`.
- ☐ I intentionally created and resolved a merge conflict.
- ☐ I inspected branch history with `git log --oneline --graph --all`.

Deliverable: a repository with a merged feature and a conflict-resolution commit in its history.

Day 3 — GitHub collaboration: Issues and Pull Requests

Outcome: use GitHub's collaboration workflow instead of pushing every change directly to `main`.

Time	Learn this → Do this
0:00–0:20	**GitHub repository pages; `clone`; local vs remote.** → Clone your repository into a second folder and inspect `git remote -v`.

0:20–0:40	**Issues, labels, and task descriptions.** → Open an Issue describing one small improvement, with a clear goal and acceptance criteria.
0:40–1:00	**Issue-linked branches and commits.** → Create a branch named `feature/<short-description>` and make the change. Reference the Issue in the commit or PR description (for example, `Closes #1`).
1:00–1:25	**Pull Requests (PRs): base, compare, review, merge.** → Push the branch and open a PR into `main`. Write what changed, why, and how you tested it.
1:25–1:45	**Reviewing changes.** → Read the Files changed tab, leave a review comment on your own PR, make a follow-up commit, and confirm the PR updates.
1:45–2:00	**Merge and cleanup.** → Merge the PR on GitHub, pull the updated `main` locally, and delete the merged remote branch if GitHub has not done so.

Day 3 checklist

- ☐ I cloned a repository and checked its remote URL.
- ☐ I created an Issue with acceptance criteria.
- ☐ I pushed a feature branch to GitHub.
- ☐ I opened a Pull Request and described the change and test steps.
- ☐ I reviewed the diff, made a follow-up change, and merged the PR.
- ☐ I synchronized my local `main` with GitHub.

Deliverable: one closed Issue and one merged Pull Request linked to it.

Day 4 — Syncing, undoing, and recovering safely

Outcome: synchronize work and choose the right recovery command without losing changes accidentally.

Time	Learn this → Do this
0:00–0:20	**`fetch` vs `pull`; `push`; remote-tracking branches.** → Run `git fetch`, inspect `git status` and `git log --oneline --all`, then use `git pull` when ready to integrate remote changes.
0:20–0:40	**Push rejection and synchronization.** → Make a change through GitHub's web editor, then locally try a push from a stale branch; fetch/pull, resolve any issue, and push again.
0:40–1:00	**`stash`, `stash list`, `stash pop`.** → Make an uncommitted change, stash it, switch branches, return, and restore the work.
1:00–1:25	**`restore`, `revert`, and `reset`.** → In a disposable practice repository, discard an unstaged edit with `restore`, undo a committed change with `revert`, and compare `reset --soft` with `reset --mixed`.
1:25–1:45	**Merge vs rebase (conceptual introduction).** → Draw the commit graph for each, then try `git rebase main` on a throwaway feature branch and inspect the result.
1:45–2:00	**Recovery review.** → Use `git reflog` to inspect recent HEAD movements. Write down which command you would use to undo a published commit safely.

Day 4 checklist

- ☐ I can explain the difference between `fetch` and `pull`.
- ☐ I synchronized a stale local branch with a remote change.
- ☐ I saved and restored uncommitted work with `stash`.
- ☐ I practiced `restore`, `revert`, and `reset` in a disposable repository.
- ☐ I understand that `revert` is usually safer for undoing a commit that has already been shared.
- ☐ I inspected `reflog` and understand what it records.

Deliverable: a short `notes/day-04-recovery.md` explaining `restore` vs `revert` vs `reset`, and `fetch` vs `pull`.

> **Safety:** never experiment with `git reset --hard`, force-push, or history rewriting in a repository containing work you care about. Practice destructive commands only in a disposable repository. Avoid force-pushing shared branches unless your team explicitly agrees.

Day 5 — A complete, professional GitHub workflow

Outcome: complete a small project from task to release using the workflow you learned.

Time	Learn this → Do this
0:00–0:20	**Good commit messages and a clean history.** → Review your history; make messages specific and action-oriented (for example, `Add profile section`). Learn the idea behind Conventional Commits, but keep one consistent style.
0:20–0:40	**Repository documentation and hygiene.** → Improve `README.md`; add an appropriate license if you intend to publish the project; review `.gitignore`; never commit passwords, API keys, tokens, or private data.
0:40–1:00	**Tags and GitHub Releases.** → Create an annotated tag such as `v1.0.0`, push it with `git push origin v1.0.0`, and create a GitHub Release from the tag.
1:00–1:45	**Capstone workflow.** → Open an Issue, create a feature branch, make 2–3 focused commits, push it, open a PR, review the diff, merge it, and pull the updated `main`.

1:45–2:00	**Final audit and reflection.** → Check the README, Issues, PRs, branches, tags, and commit graph. Write 5 Git commands you can now use without looking them up and 3 topics to practice next.

Day 5 checklist

- ☐ I reviewed and improved my commit messages.
- ☐ My repository has a clear README and a suitable `.gitignore`.
- ☐ I checked that no secrets or private files are committed.
- ☐ I created and pushed a version tag.
- ☐ I created a GitHub Release.
- ☐ I completed the full Issue → branch → commits → PR → review → merge workflow.
- ☐ I wrote a short reflection on what I learned and what to practice next.

Deliverable: a polished GitHub repository with documentation, a complete merged PR workflow, and a `v1.0.0` release.

Quick command reference

Goal	Command
Check state	`git status`
Stage a file	`git add <file>`
Stage all current changes	`git add .`
Review changes	`git diff` and `git diff --staged`
Commit	`git commit -m "Describe the change"`
View history	`git log --oneline --graph --all`

Create/switch branch	`git switch -c feature/name`
Switch branch	`git switch main`
Merge branch	`git merge feature/name`
Add remote	`git remote add origin <repo-url>`
View remotes	`git remote -v`
Push first time	`git push -u origin main`
Push branch	`git push -u origin <branch>`
Download remote updates	`git fetch`
Download and integrate	`git pull`
Temporarily save changes	`git stash` / `git stash pop`
Undo a published commit safely	`git revert <commit>`
Inspect recent HEAD history	`git reflog`
Create annotated tag	`git tag -a v1.0.0 -m "Version 1.0.0"`

After these 5 days

You should be able to create and publish repositories, make and inspect commits, branch and merge, resolve basic conflicts, collaborate through Issues and Pull Requests, synchronize remote work, and recover from common mistakes. Keep practicing with a real project: Git becomes natural through repeated use, not memorization.

Suggested next topics: interactive rebase, cherry-pick, submodules, Git hooks, branch protection rules, GitHub Actions, and release automation.